INTRODUÇÃO À LINGUAGEM DE PROGRAMAÇÃO C

Objetivos

- Apresentar a descrição da linguagem C;
- Apresentar as estruturas básicas de controle em C;
- Apresentar a forma de codificação em linguagem C;
- Apresentar padrões de mapeamento para a linguagem C.

Histórico

- 1972 – primeira versão de C , por Dennis Ritchie;
- 1979 – publicação do livro "The C Programming Language" (Brian Kernigham e Dennis Ritchie);
- 1989 – padronização ANSI C e ISO C (C89)
- 1999 – padronização C99
- 2011 – padronização C11
- 2017 – padronização C18
- 2022 – padronização C23

Descrição da linguagem

- Alfabeto

  Um programa em C poderá conter os seguintes caracteres:

  - as vinte e seis (26) letras do alfabeto inglês:
    A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
    a b c d e f g h i j k l m n o p q r s t u v w x y z

  - os dez (10) algarismos:
    0 1 2 3 4 5 6 7 8 9

  - os símbolos:

<	menor	( )	parênteses
>	maior	[ ]	colchete
.	ponto	{ }	chaves
,	vírgula	+	soma
:	dois pontos	-	subtração
;	ponto-e-vírgula	*	asterisco
=	igualdade	/	barra
!	exclamação	#	sustenido
?	interrogação	“	aspas
&	ampersete ("e" comercial)	‘	apóstrofo
^	circunflexo	%	porcento
|	barra em pé	~	til

- Pontuação
  - Ponto-e-vírgula é usado para separar comandos, a menos que outro separador seja necessário;
  - Em alguns casos de operadores, convém o uso de espaços em branco antes, e depois.

- Observação:
  Em C utilizam-se, ***obrigatoriamente***, as letras minúsculas para os comandos próprios da linguagem. Recomenda-se não utilizar acentos ou cedilha.

Tipos de dados

- Principais tipos básicos

Algoritmo	C
inteiro	**int**
real	**double**
caractere	**char**

- Outros tipos:

short	inteiros "*curtos*"
long	inteiros "*longos*": [-2.147.483.648,2.147.483.647]
signed	inteiros com sinal: [-32768, 32767]
unsigned	inteiros sem sinal: [0, 65535]
float	reais com precisão simples

- Especificação de classe de armazenamento:

auto	alocação na pilha até retorno de função
static	alocação em memória durante a execução
register	alocação em registrador, se possível (valores escalares)
extern	não faz alocação de memória (declaração externa)

- Qualificativos de tipos:

const	constante
volatile	volátil (usada em programação do tipo ***multi-threaded***)

Exemplos:

```
int          y;
static int   x;
unsigned     z;
float        i, j, k;
double       a, b, c;
char         letra;
```

- Apontadores

  Se o nome de uma variável for precedido por um asterisco (*), isso significará que o objeto será do tipo apontador.

  Podem ser operados aritmeticamente, como inteiros, e comparados com outros apontadores, ou a uma constante desse tipo, ***NULL*** (que não aponta para nenhum objeto válido).

  Exemplos:

```
int      *y;
double   *i, j, k;
char     *letra;
```

  O endereço de um objeto poderá ser manipulado diretamente precedendo o nome do objeto por um sinal & (*ampersete*).

  Exemplos:

```
i      = &j;
nome = &endereco;
```

- Definição de novos tipos

  Forma geral:

  typedef <nome> <tipo>;

  Exemplos:

```
typedef   REAL      double;
typedef   INTEIRO   int   ;

REAL      a, b, c;
INTEIRO   *i, j;
```

- Constantes

  - Constante inteira

    Exemplos:

```
10  , 532 , -10567
067, 05421, 0724            (octal)
0L  , 130000000L, 3241L     (inteiro longo)
0x41, 0xFFFF, 0xA0          (hexadecimal)
```

    Observação:
    Em geral, tem-se a faixa de -32768 a +32767, para 2 bytes de representação.

  - Constante real

    Exemplos:

```
10.465   -5.61   +265.   0.0f   0.0   .731   .37e+2   -3.e-1
```

    Observações:
    A vírgula decimal é representada por ponto decimal.

Em geral, tem-se a faixa de 10^{-38} a 10^{+38} (para ***float***) e 10^{-308} a 10^{+308} (para ***double***).

- Constante literal

  Exemplos:
  Caractere : '1', ' ', '*', 'A', 'a', '?'
  Cadeia : "BRANCO", "CEU AZUL"

  Observações:
  O tamanho da cadeia é limitado.
  As cadeias são terminadas pelo símbolo especial '\0'.

- Caracteres predefinidos:

'\0'	nulo (fim de cadeia de caracteres)
'\n'	passa para a próxima linha
'\t'	passa para a próxima coluna de tabulação (9,17, ...)
'\b'	retorna o cursor uma coluna
'\r'	posiciona o cursor no inicio da linha
'\f'	limpa a tela ou passa para a próxima página
'\\'	barra invertida
'\''	apóstrofo
'\"'	aspas

- Definição de constantes

  Formas gerais:

  #**define** <NOME1> <valor1>
  #**define** <NOME2> (<valor2>)

  const <tipo> <NOME> = <valor>;

  Exemplos:
  #**define** TRUE 1
  #**define** FALSE 0

  const float PI = 3.1415926**f**;

- Variáveis

  - Nome de variável

    a) O nome de uma variável tem tamanho determinado;
    b) O primeiro caractere é uma letra ou travessão (_);
    c) Outros caracteres podem ser letra, algarismo ou travessão (_).

    Exemplos:
    Nomes válidos : I, a, de, V9a, Lista_Notas

    Nomes inválidos: x+, t.6, 43x, so 5

- Declaração de variáveis
  - Variáveis simples

    Forma geral:

    ```
    <tipo 1> <lista de nomes 1>;
    <tipo 2> <lista de nomes 2>;
      ...
    <tipo N> <lista de nomes N>;
    ```

    Exemplos:

    ```
    char    fruta;
    int     i, j, k;
    double  p, DELTA;
    ```

    A declaração de variáveis pode ser usada também para a atribuição de valores iniciais.

    Exemplos:

    ```
    int   x = 10,
          y = 20;
    ```

- Variáveis agrupadas
  - Homogêneas

    Forma geral:

    ```
    <tipo 1> <lista de nomes 1> [índice];
    <tipo 2> <lista de nomes 2> [índice 1] [índice 2];
      ...
    <tipo N> <lista de nomes N> [índice] ... ;
    ```

    Exemplos:

    ```
    char  frutas   [10],
    char  letras   [3] = {'a','b','c'},
    char  nomes [3][10] =
          {
            "Alfredo",
            "Jose",
            "Mario"
          };
    int   v[5] = {1,2,3,4,5}, j[4][4], k;
    ```

    Observação:

    O primeiro elemento tem índice igual a zero.

- Heterogêneas

  Forma geral:

  enum {<lista de valores>} <declaração de nomes>;

  struct <nome> {<campos>} <lista de nomes>;

  union <nome> {<lista>} <lista de nomes>;

  Exemplos:

```
enum {banana,laranja,abacaxi} fruta = banana;

struct pessoa
{
  char nome, endereco;
  int   rg, cpf, titulo_eleitoral;
};

struct pessoa funcionario, operario;
```

  Observação:

  O acesso aos campos de uma estrutura pode ser feito por

  nome.membro ou apontador -> membro

- Tipos de operadores

  - Aritméticos

Algoritmo	C
* / ***div mod***	* / %
+ -	+ -

  Observações:

  O operador ***div*** (divisão inteira) é a própria barra (/), quando os operandos forem inteiros.

  Existem formas compactas para incremento e decremento:

<variável inteira>++	pós-incremento
++<variável inteira>	pré-incremento
<variável inteira>--	pós-decremento
--<variável inteira>	pré-decremento

  - Relacionais

Algoritmo	C
< ≤ > ≥	< <= > >=

= ≠	== !=

Observação:

O resultado de uma comparação de dois valores pode ser 0 (falso) ou 1 (verdadeiro).

- Lógicos (bit a bit)

Algoritmo	C
complemento de um	~
e	&
ou-exclusivo	^
ou	|
deslocamento à direita	>>
deslocamento à esquerda	<<

Observação:
O resultado de uma operação lógica é um valor cujos bits são operados um a um de acordo com a álgebra de proposições.

- Conectivos lógicos

Algoritmo	C
não	!
e	&&
ou	||

- Prioridade de operadores

Operador	Associação
( ) [ ] { } _++ _-- ->	à esquerda
! ~ ++_ --_ + - (tipo) * & **sizeof**	à direita
* / %	à esquerda
+ -	à esquerda
>> <<	à esquerda
< <= >= >	à esquerda
== !=	à esquerda
&	à esquerda
^	à esquerda
|	à esquerda
&&	à esquerda
||	à esquerda
?:	à direita
= += -= *= /= %=	à direita
>>= <<= &= |= ^=	à direita
,	à esquerda

- Funções intrínsecas

As regras usadas na formação dos nomes dessas funções intrínsecas são as mesmas utilizadas para os nomes das variáveis.

Exemplo:

```
a  =  sin ( b )
```

a - nome da variável que receberá o resultado da função;
sin - função (seno) predefinida em C ;
b - nome da variável que vai ser o argumento da função.

Principais funções matemáticas (**math.h**)

Tipo	Nome (argumento)	Tipo de argumento	Descrição
double	sin (X)	**double**	seno (em radianos)
double	cos (X)	**double**	cosseno (em radianos)
double	atan(X)	**double**	arco tangente
double	sqrt (X)	**double**	raiz quadrada
double	exp (X)	**double**	exponencial de "e"
int	abs (X)	**int**	valor absoluto inteiro
double	fabs(X)	**double**	valor absoluto real
double	log (X)	**double**	logaritmo neperiano (base “e”)
double	log10 (X)	**double**	logaritmo base 10
double	pow(X,Y)	**double, double**	elevar X a Y

Principais funções literais (**string.h**)

Tipo	Nome (argumento)	Tipo de argumento	Descrição
size_t	strlen (X)	**char ***	comprimento da cadeia X
char *	strcpy (X, Y)	**char *, char ***	copiar a cadeia Y para X
char *	strdup (X)	**char ***	duplicar a cadeia X
char *	strcat (X, Y)	**char *. char ***	concatenar X com Y
int	strcmp (X, Y)	**char *, char ***	comparar as cadeia X e Y resultado significado < 0 X precede Y = 0 X igual a Y > 0 X sucede Y

A linguagem C dispõe de uma significativa biblioteca básica, com diversas funções além das citadas acima.

- Expressões
  - Aritmética

    Exemplos:

Algoritmo	C
10 + 15	10 + 15
543.15/3	543.15 / 3
(x + y + z)*a/z	((x + y + z) * a) / z

  - Lógica

    Exemplos:

Algoritmo	C
A = 0	A == 0
a $\neq$ 1	a != 1
(A $\geq$ 0) & (a $\leq$ 1)	(A >= 0) && (a <= 1)

    Observação:

    Para efeito de clareza, ou para mudar a precedência de operadores, pode-se separar as proposições por parênteses.

- Estrutura de programa

```
// definições para pré-processamento

// definições globais

// definições de funções e procedimentos
  <tipo> <nome> (<lista de parâmetros>)
  {
     // definições locais

     // comandos
  }

// parte principal
    int main ( int argc, char *argv [ ] )
  {
     // definições locais

     // comandos
     return 0;            // ou return EXIT_SUCCESS;
  }

// definições de funções e procedimentos (OPCIONAL)
  <tipo> <nome> (<lista de parâmetros>)
  {
     // definições locais

     // comandos
  }
```

- Comentários

  Comentários são precedidos pelos sinais // , ou /* */ envolvendo o texto.

  Exemplo:

```
int main ( )
{
   // Este programa nao faz nada - comentario

   /*
      que também pode ser colocado assim
   */
   return 0;
}
```

- Atribuição
  - Atribuição simples

    Forma geral:

    <variável> = <expressão>;

    Exemplo:
    x = 0 ;
    a = 1.57;
    letra = ‘A’ ;

  - Atribuição múltipla

    Forma geral:

    <variável 1> = <variável 2> = <expressão>;

    Exemplo:
    x = y = 0;

    Observação:

    A execução inicia-se pela direita.

  - Atribuição composta

    Forma geral:

    <variável> <operador> = <expressão>;

    Exemplo:
    i += 1 ou i = i + 1

    Observação:

    Operadores permitidos: + - * / % >> << | & ^

  - Atribuição condicional

    Forma geral:

    <variável> = <teste> ? <expressão 1>: <expressão 2>;

    Exemplo:
    x = (a < b) ? a: b;

- Descrição de entrada e saída
  - Entrada/Saída formatada (padrão C):

    Forma geral:

    scanf (<formato>,<lista de apontadores>);
    printf (<formato>, <lista de itens>);

    Observação:

    É necessário usar a definição abaixo (ou similar):

    #**include** <stdio.h>

  - Especificação de formatos:

    Forma geral:

    %<sinais><<0><largura>>< . ><precisão><conversão>

    onde:
    <sinais> podem ser:
    '-' - alinha a esquerda a saída
    '+' - conversão de sinal (+ ou -)
    ' ' - conversão de sinal (" " ou -)
    '#' - começo com (0, 0x onde apropriado)

    <0> - preenchimento com zeros
    <largura> - largura mínima do campo
    <precisão> - número máximo de caracteres
    - ou número de dígitos fracionários
    (neste caso é precedida por < . >)
    <conversão> - pode ser:

caractere	argumento	conversão
d	**int**	para decimal
i	**int**	para inteiro
o	**int**	para octal
x	**int**	para hexadecimal
u	**int**	para decimal, sem sinal
c	**char/int**	para um caractere
s	**char ***	para cadeia de caracteres
e	**float**	para real com expoente
f	**float**	para real sem expoente
g	**float**	para real
%		sinal %
ld	**long**	para decimal
lo	**long**	para octal
lx	**long**	para hexadecimal
le	**double**	para real com expoente
lf	**double**	para real sem expoente
p	endereço	para endereço (&) na memória

Exemplos:

%5c - X do tipo caractere e com valor igual a 'A'					A
%5d - X do tipo inteiro e com valor igual a 100			1	0	0
%5.2f - X do tipo real e com valor igual a -1	-	1	.	0	0

Observação:

Se a largura (no exemplo, 5) não for suficiente para conter o número na sua forma de representação interna, o tamanho padrão para cada tipo será usado.

- Caracteres com funções especiais em formatos:

caractere	função
\0	fim da cadeia de caracteres
\n	fim de linha (LF)
\t	tabulação
\b	retrocesso (BS)
\r	retorno de carro (CR)
\f	avanço de carro (FF)
\\	barra invertida
\'	apóstrofo
\”	aspas
\nnn	representação em um ***byte*** de valor octal
\xnn	representação em um ***byte*** de hexadecimal
‘\unnnn’	representação em dois ***bytes*** de caractere ***Unicode***

Exemplo completo de programa:

```
#include <stdio.h>

int main ( void )
{
   int i, j;

   printf ( "Exemplo: " );
   printf ( "\n" );
   printf ( "Fornecer um valor inteiro: " );
   scanf ( “%d”, &j );
   i   =   j * 2 + 10;
   printf ( "%s %d\n","O resultado e\' igual a ", i );
   printf ( “\nPressionar <Enter> para terminar.” );
   getchar ( );
   return ( 0 );
}
```

Se fornecido o valor 5 para a variável ***j*** , o resultado será:

```
O resultado e' igual a 20
```

- Estruturas de controle
  - Sequência simples

    Forma geral:

Algoritmo	C
<comando>	<comando> ;
<comando>	<comando> ;

    Observação:
    Em C todos os comandos são separados por ponto-e-vírgula.

- Estrutura alternativa

  - Alternativa simples

    Forma geral:

Algoritmo	C

se <condição>
então
  <comandos>
fim se
	if (<condição>)
	{
	  <comandos> ;
	}

  - Alternativa dupla

    Forma geral:

Algoritmo	C
se <condição> então <comandos 1> senão <comandos 2> fim se	**if** (<condição>) { <comandos 1> ; } **else** { <comandos 2> ; }

  - Alternativa múltipla

    Forma geral:

Algoritmo	C

escolher <valor>
 	
<opção 1>:
  <comandos 1>
 	
<opção 2>:
<opção 3>:
  <comandos 2>
 	
. . .
<opção n-1>:
  <comandos N-1>
 	
senão
  <comandos N>
 	
fim escolher
	switch <valor>
	{
	  case 1:
	  <comandos 1> ;
	  break;
	  case 2:
	  case 3:
	  <comandos 2> ;
	  break;
	. . .
	  case (n-1):
	  <comandos N-1> ;
	  break;
	default:
	  <comandos N>;
	  break;
	}

    Observações:

    A variável de decisão deve ser de tipo escalar.

    Se o comando ***break*** for omitido, os comandos da próxima opção também serão executados.
    A indicação ***default*** é opcional.

- Estrutura repetitiva

  - Repetição com teste no início

    Forma geral:

Algoritmo	C
repetir enquanto <condição>	**while** (<condição>)
	{
<comandos>	<comandos> ;
fim repetir	}

    Observação:
    A condição para execução é sempre verdadeira.

  - Repetição com teste no início e variação

    Forma geral:

Algoritmo	C
repetir para	**for**(
<variável> = <valor inicial>	<valor inicial> ;
: <fim>	<teste de fim> ;
: <variação>	<variação>)
	{
<comandos>	<comandos> ;
fim repetir	}

    Observações:
    A condição para execução é sempre verdadeira.
    Em C , qualquer um dos elementos, ou mesmo todos, podem ser omitidos. Entretanto, se tal for preciso, recomenda-se o uso de outra estrutura mais apropriada.

- Repetição com teste no fim

    Forma geral:

Algoritmo	C
repetir até <condição>	**do**
	{
<comandos>	<comandos> ;
fim repetir	} **while** (<condição>) ;

    Observação:
    A condição para execução é sempre verdadeira.

- Interrupções

  Em C , as repetições podem ser interrompidas, em sua sequência normal de execução através dos comandos:

  break; e **continue**;

  O comando ***break*** serve para interromper completamente uma repetição, passando o controle ao próximo comando após a estrutura repetitiva.
  O comando ***continue*** interrompe uma iteração, voltando ao início da mesma.